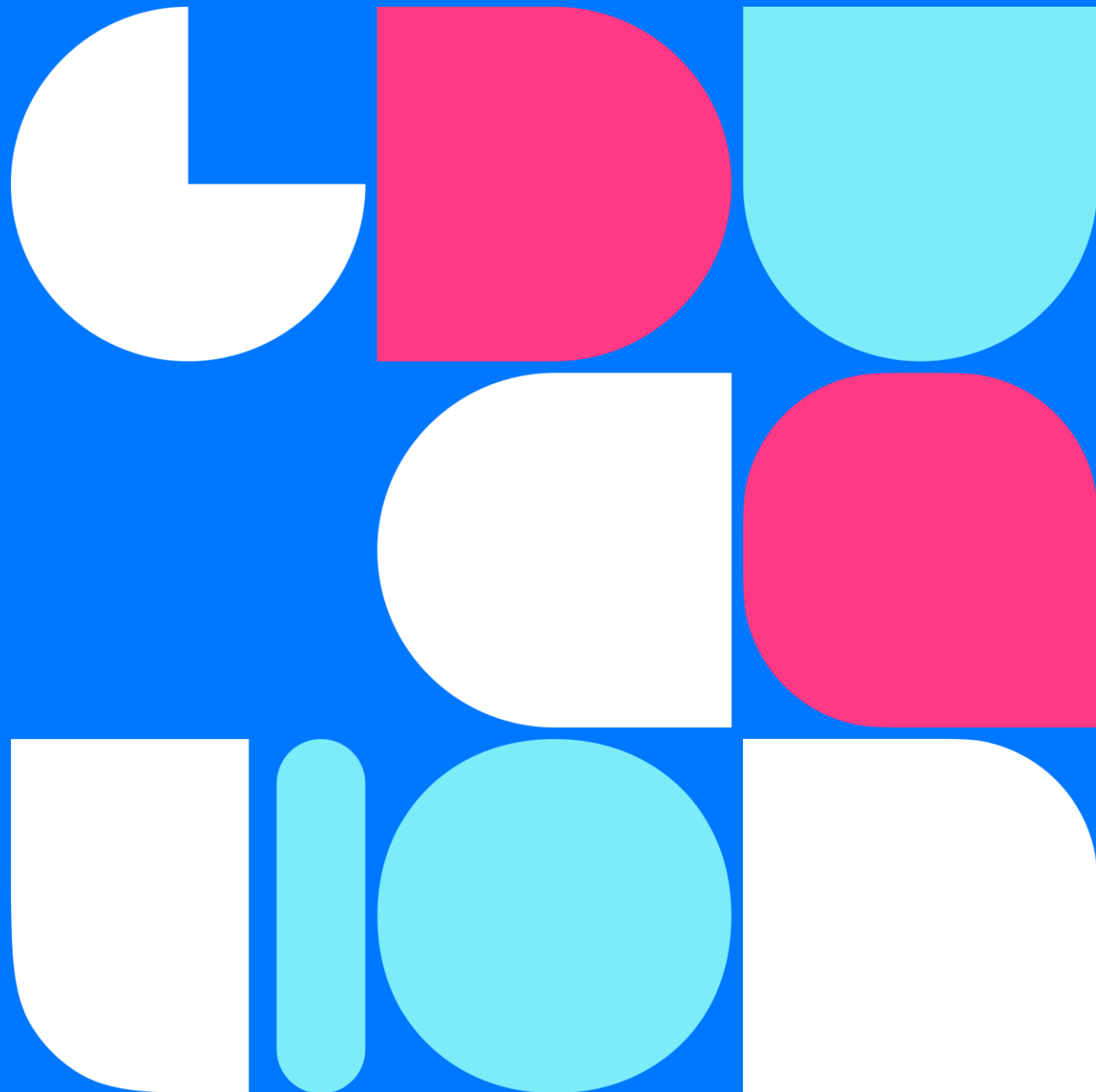




JaCoCo gradle plugin for Android

Линейский Аким



Покрытие кода



Покрытие кода

Покрытие кода измеряет, какие части исходного кода были проверены тестами, а какие остаются уязвимыми для ошибок.

- Цель - Оценка качества тестирования и выявление непроверенных участков кода
- Метрики – строки, ветки, методы, классы, пакеты

app > vk.autotest.project.noteData > Note.java Sessions

Note.java

```
1. package vk.autotest.project.noteData;
2.
3. import java.util.Date;
4.
5. public class Note {
6.     private String id;
7.     private String title;
8.     private String content;
9.
10.    private final Date createdAt;
11.
12.    public Note(String id, String title, String content) {
13.        this.id = id;
14.        this.title = title;
15.        this.content = content;
16.        this.createdAt = new Date();
17.    }
18.
19.    public String getId() { return id; }
20.    public String getTitle() { return title; }
21.    public String getContent() { return content; }
22.
23.    public void setId(String id) {
24.        this.id = id;
25.    }
26.
27.    public void setTitle(String title) {this.title = title;}
28.
29.    public void setContent(String content) {this.content = content;}
30.    public Date getCreatedAt() {return createdAt;}
31. }
```

Created with JaCoCo 0.8.8.202204050719

app

Sessions

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
vk.autotest.project		0 %		0 %	8 8	29 29	6 6	2 2
vk.autotest.project.noteView		56 %		66 %	8 16	19 44	7 14	2 4
vk.autotest.project.noteData		91 %		100 %	2 14	3 26	2 12	0 2
Total	223 of 387	42 %	5 of 11	54 %	18 38	51 99	15 32	4 8

Created with JaCoCo 0.8.8.202204050719

JaCoCo

Java Code Coverage - (Покрытие кода Java)

- Разработан как открытая альтернатива EMMA, поддерживается Eclipse Foundation.
- Измеряет покрытие в режиме on-the-fly (без модификации байт-кода).

Создан для Java и Android, но интегрируется с Kotlin, Groovy и другими JVM-языками

Поддерживается в Gradle, Maven, Ant и CI-системах (Jenkins, GitHub Actions).

Согласно опросу *2023 JVM Ecosystem Report*, JaCoCo используют 62% Java-разработчиков для анализа покрытия.



GitHub Actions

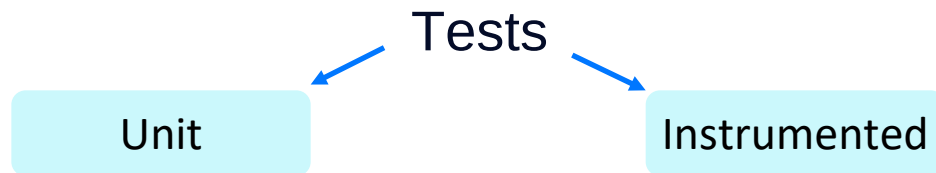


JaCoCo, Gradle, Android

Gradle – сборщик проекта

JaCoCo agent - Встраивается в JVM во время выполнения тестов

App – выполняемая задача Gradle



Принцип работы:

- Модифицирует байт-код специальными метками
- Собирает данные о выполнении каждой строки
- Собирает отчет о покрытии кода



Установка JaCoCo

```
1  apply plugin: 'com.android.application'
2  apply plugin: 'jacoco'

16  jacoco {
17      toolVersion = "0.8.12"
18  }

48  tasks.withType(Test).configureEach { Test it ->
49      jacoco.excludes = ['jdk.internal.*', '**/*']
50  }
```

 build.gradle (:app)

```
19  android {
20      namespace 'vk.autotest.project'
21      compileSdkVersion 35
22
23      defaultConfig { DefaultConfig it ->
24          applicationId 'vk.autotest.project'
25          minSdkVersion 24
26          targetSdkVersion 35
27          versionCode 1
28          versionName "1.0"
29          testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
30      }
31      compileOptions { CompileOptions it ->
32          sourceCompatibility JavaVersion.VERSION_17
33          targetCompatibility JavaVersion.VERSION_17
34      }
35
36      buildTypes { NamedDomainObjectContainer<BuildType> it ->
37          debug {
38              testCoverageEnabled true
39          }
40          release {
41              minifyEnabled false
42              proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
43                  'proguard-rules.pro'
44          }
45      }
46  }
```

JaCoCo Report Task

 build.gradle (:app)

Task to report

```
52 task jacocoUITestReport(type: JacocoReport, dependsOn: ['connectedDebugAndroidTest']) {  
53     reports {  
54         xml.required = true  
55         html.required = true  
56     }  
57  
58     def fileFilter = ['**/R.class', '**/R$.class', '**/BuildConfig.*',  
59         '**/Manifest.*', '**/*Test.*', 'android/**/*.*']  
60     def debugTree : ConfigurableFileTree = fileTree(dir: "$buildDir/intermediates/javac/debug",  
61         excludes: fileFilter)  
62  
63     sourceDirectories.setFrom(files(["src/main/java"]))  
64     classDirectories.setFrom(files([debugTree]))  
65     executionData.setFrom(fileTree(dir: buildDir, includes: [  
66         'outputs/code_coverage/debugAndroidTest/connected/*/coverage.ec']))  
67     doLast {println "JaCoCo UI report generated"}  
68 }
```

*UI instrumented
test report*

Unit test report

```
83     executionData.setFrom(fileTree(dir: buildDir, includes: [  
84         'outputs/unit_test_code_coverage/debugUnitTest/testDebugUnitTest.exec']))  
85     doLast {println "JaCoCo UNIT report generated"}  
86 }
```

Тестирование



Тестирование

Unit tests

- Тестирование отдельных классов, методов, алгоритмов
- Скорость
- Запуск на JVM, а не на конкретном Android

```
@Test
public void deleteNoteInvalidIdShouldNotAffectList() {
    NotesRepository repo = new NotesRepository();
    repo.addNote(new Note( id: "1", title: "Note1", content: "Content1"));
    repo.deleteNote( id: "invalid_id");
    assertEquals( expected: 1, repo.getAllNotes().size());
}
```

Instrumented tests

- Тестирование взаимодействия компонентов
- Запуск на машине устройства
- Взаимодействие с Android (Activity, Frame)

```
@Test
public void testFabOpensDialog() {
    onView(withId(R.id.addNoteFab)).perform(click());
    onView(withId(R.id.editTextTitle)).check(matches(isDisplayed()));
    onView(withId(R.id.editTextContent)).check(matches(isDisplayed()));
    onView(withId(R.id.saveButton)).check(matches(isDisplayed()));
}
```

Unit test

@Before

```
public void setUp() {  
    viewModel = new NotesViewModel();  
    testNotes = Arrays.asList(  
        new Note( id: "1", title: "Banana", content: "Content 1"),  
        new Note( id: "2", title: "Apple", content: "Content 2"),  
        new Note( id: "3", title: "Cherry", content: "Content 3")  
    );  
    for (Note note : testNotes) {  
        viewModel.addNote(note.getTitle(), note.getContent());  
    }  
}
```

@Test

```
public void testSortByTitleAscending() {  
    viewModel.setSortType(NotesViewModel.SortType.BY_TITLE);  
    List<Note> sortedNotes = getNotesFromLiveData(viewModel.getNotes());  
  
    assertEquals( expected: "Apple", sortedNotes.get(0).getTitle());  
    assertEquals( expected: "Banana", sortedNotes.get(1).getTitle());  
    assertEquals( expected: "Cherry", sortedNotes.get(2).getTitle());  
}
```

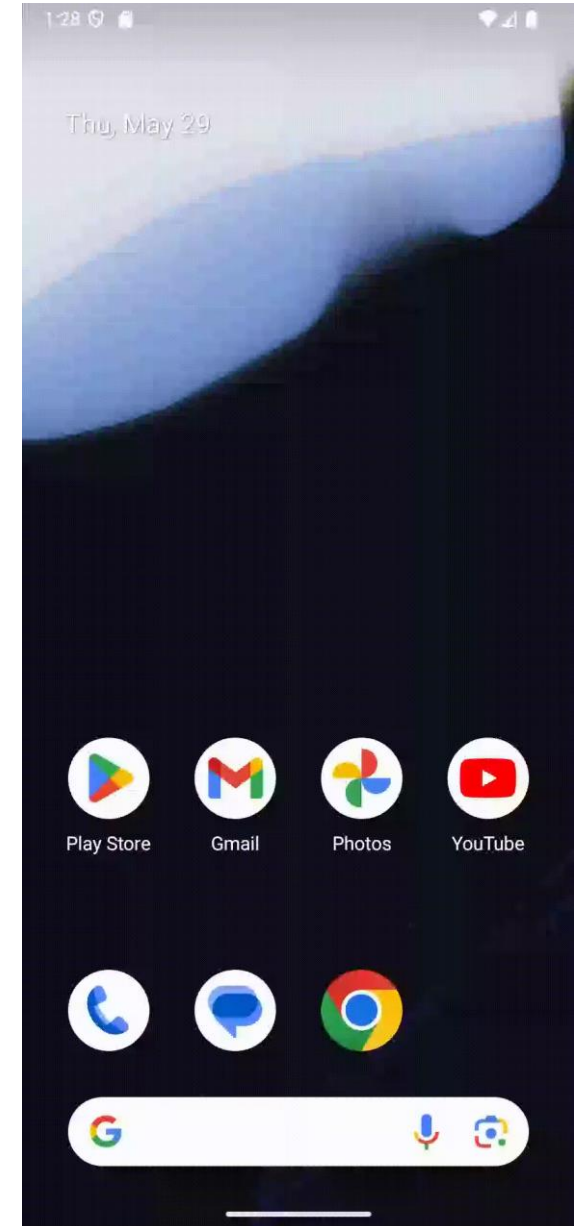
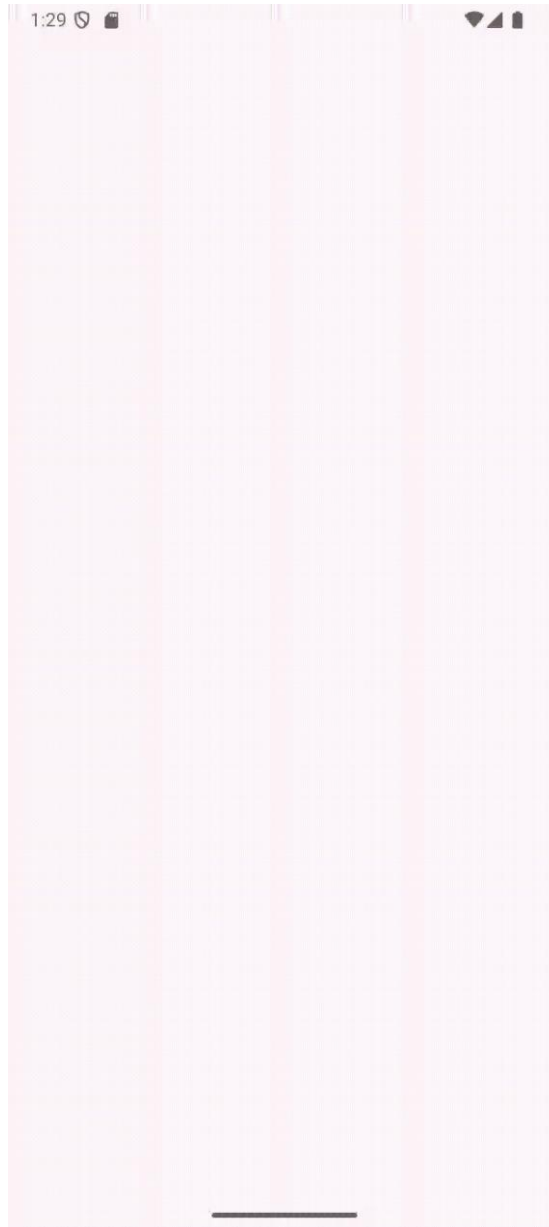
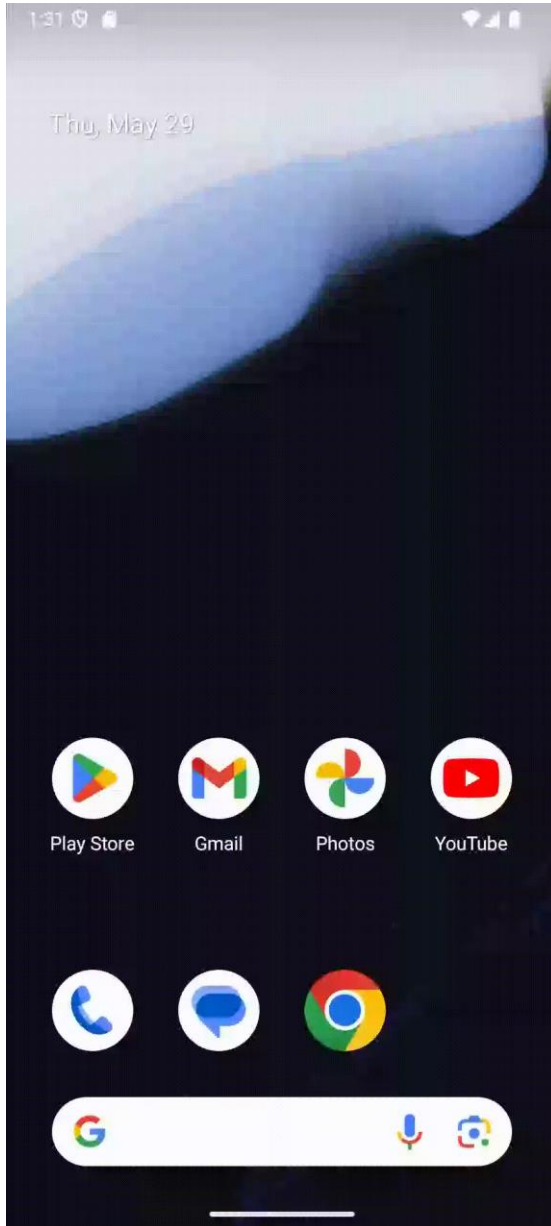
@Test

```
public void deleteNoteInvalidIdShouldNotAffectList() {  
    NotesRepository repo = new NotesRepository();  
    repo.addNote(new Note( id: "1", title: "Note1", content: "Content1"));  
    repo.deleteNote( id: "invalid_id");  
    assertEquals( expected: 1, repo.getAllNotes().size());  
}
```

@Test

```
public void getAllNotesShouldReturnCopy() {  
    NotesRepository repo = new NotesRepository();  
    Note note = new Note( id: "1", title: "Note", content: "Test");  
    repo.addNote(note);  
    List<Note> notes = repo.getAllNotes();  
    notes.clear();  
    assertEquals( expected: 1, repo.getAllNotes().size());  
}
```

UI test (instrumented)



Покрытие кода



Отчет

Типы покрытий в отчете:

- *Instruction* - байт-код инструкции
- *Branch* - возможные пути
- *Line* -выполненные строки кода
- *Method* - процент вызванных методов у класса
- *Class* - процент вызовов классов в пакете

app > vk.autotest.project > AddNoteDialog.java

Sessions

AddNoteDialog.java

```
1. package vk.autotest.project;
2.
3. import android.os.Bundle;
4. import android.view.*;
5. import android.widget.*;
6. import androidx.annotation.*;
7. import androidx.fragment.app.DialogFragment;
8. import androidx.lifecycle.ViewModelProvider;
9.
10. import vk.autotest.project.noteView.NotesViewModel;
11.
12. public class AddNoteDialog extends DialogFragment {
13.
14.     @Nullable
15.     @Override
16.     public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
17.         View view = inflater.inflate(R.layout.dialog_add_note, container, false);
18.         EditText titleInput = view.findViewById(R.id.editTextTitle);
19.         EditText contentInput = view.findViewById(R.id.editTextContent);
20.         Button saveButton = view.findViewById(R.id.saveButton);
21.
22.         NotesViewModel viewModel = new ViewModelProvider(requireActivity()).get(NotesViewModel.class);
23.
24.         saveButton.setOnClickListener(v -> {
25.             String title = titleInput.getText().toString();
26.             String content = contentInput.getText().toString();
27.             if (title.isEmpty()) {
28.                 Toast.makeText(getContext(), "Заголовок не может быть пустым", Toast.LENGTH_SHORT).show();
29.             } else if (title.length() > 50) {
30.                 Toast.makeText(getContext(), "Заголовок слишком длинный", Toast.LENGTH_SHORT).show();
31.             } else {
32.                 viewModel.addNote(title, content);
33.                 dismiss();
34.             }
35.         });
36.         return view;
37.     }
38. }
```

Сравнение покрытий

app > vk.autotest.project.noteView > NotesViewModel.java

NotesViewModel.java

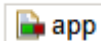
```
1. package vk.autotest.project.noteView;
2.
3. import androidx.lifecycle.*;
4. import java.util.*;
5.
6. import vk.autotest.project.noteData.Note;
7. import vk.autotest.project.noteData.NotesRepository;
8.
9. public class NotesViewModel extends ViewModel {
10.     private final NotesRepository repository = new NotesRepository();
11.     private final MutableLiveData<List<Note>> notesLiveData = new MutableLiveData<>(new ArrayList<>());
12.     private SortType currentSortType = SortType.BY_TITLE;
13.     public void addNote(String title, String content) {
14.         Note note = new Note(UUID.randomUUID().toString(), title, content);
15.         repository.addNote(note);
16.         updateNotesList();
17.     }
18.
19.     public void deleteNote(String id) {
20.         repository.deleteNote(id);
21.         updateNotesList();
22.     }
23.
24.     private void updateNotesList() {
25.         List<Note> notes = repository.getAllNotes();
26.         switch (currentSortType) {
27.             case BY_TITLE:
28.                 notes.sort(Comparator.comparing(Note::getTitle));
29.                 break;
30.             case BY_TITLE_DESC:
31.                 notes.sort(Comparator.comparing(Note::getTitle).reversed());
32.                 break;
33.         }
34.         notesLiveData.setValue(notes);
35.     }
36.     public LiveData<List<Note>> getNotes() {
37.         return notesLiveData;
38.     }
39.
40.     public void setSortType(SortType sortType) {
41.         this.currentSortType = sortType;
42.         updateNotesList();
43.     }
44.     public enum SortType {
45.         BY_TITLE,
46.         BY_TITLE_DESC
47.     }
48. }
```

app > vk.autotest.project.noteView > NotesViewModel.java

NotesViewModel.java

```
1. package vk.autotest.project.noteView;
2.
3. import androidx.lifecycle.*;
4. import java.util.*;
5.
6. import vk.autotest.project.noteData.Note;
7. import vk.autotest.project.noteData.NotesRepository;
8.
9. public class NotesViewModel extends ViewModel {
10.     private final NotesRepository repository = new NotesRepository();
11.     private final MutableLiveData<List<Note>> notesLiveData = new MutableLiveData<>(new ArrayList<>());
12.     private SortType currentSortType = SortType.BY_TITLE;
13.     public void addNote(String title, String content) {
14.         Note note = new Note(UUID.randomUUID().toString(), title, content);
15.         repository.addNote(note);
16.         updateNotesList();
17.     }
18.
19.     public void deleteNote(String id) {
20.         repository.deleteNote(id);
21.         updateNotesList();
22.     }
23.
24.     private void updateNotesList() {
25.         List<Note> notes = repository.getAllNotes();
26.         switch (currentSortType) {
27.             case BY_TITLE:
28.                 notes.sort(Comparator.comparing(Note::getTitle));
29.                 break;
30.             case BY_TITLE_DESC:
31.                 notes.sort(Comparator.comparing(Note::getTitle).reversed());
32.                 break;
33.         }
34.         notesLiveData.setValue(notes);
35.     }
36.     public LiveData<List<Note>> getNotes() {
37.         return notesLiveData;
38.     }
39.
40.     public void setSortType(SortType sortType) {
41.         this.currentSortType = sortType;
42.         updateNotesList();
43.     }
44.     public enum SortType {
45.         BY_TITLE,
46.         BY_TITLE_DESC
47.     }
48. }
```

Сравнение отчетов покрытий



app

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 vk.autotest.project.noteData		77 %		75 %	6	14	6	26	5	12	0	2
 vk.autotest.project.noteView		93 %		33 %	3	16	4	44	1	14	0	4
 vk.autotest.project		95 %		75 %	1	8	1	29	0	6	0	2
Total	36 of 387	90 %	4 of 11	63 %	10	38	11	99	6	32	0	8



app

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 vk.autotest.project		95 %		75 %	1	8	1	29	0	6	0	2
 vk.autotest.project.noteData		91 %		100 %	2	14	3	26	2	12	0	2
 vk.autotest.project.noteView		100 %		66 %	1	16	0	44	0	14	0	4
Total	14 of 387	96 %	2 of 11	81 %	4	38	4	99	2	32	0	8

CI-CD



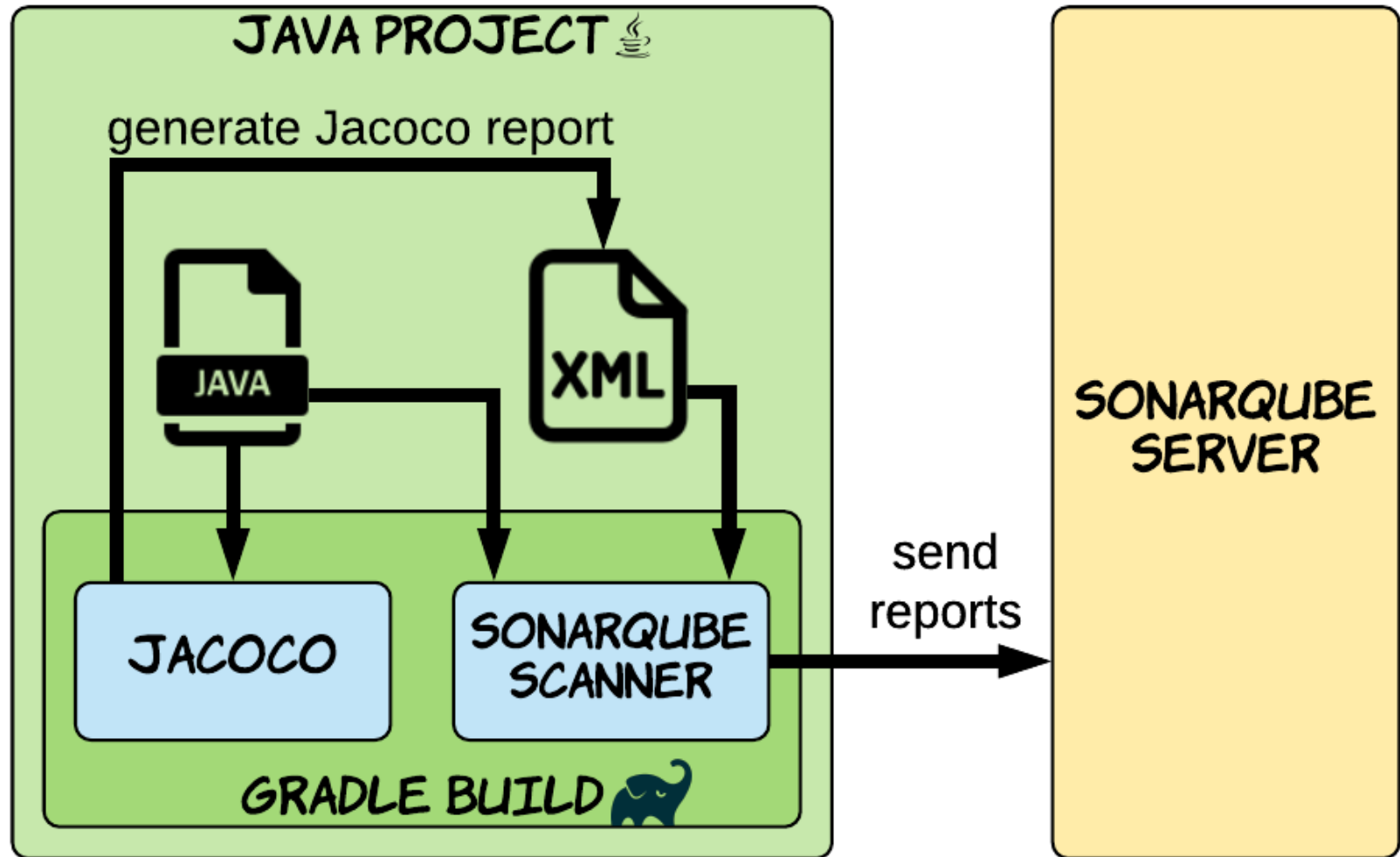
Сравнение инструментов

Инструмент	Особенности	Плюсы
JaCoCo	On-the-fly анализ, интеграция с Gradle/Maven	Быстрый, бесплатный, подробные отчеты
EclEmma	Плагин для Eclipse, на базе JaCoCo	Удобен для Eclipse-разработчиков
Cobertura	Оффлайн-анализ (post-processing)	Поддержка Groovy, простые отчеты
Clover	Коммерческий инструмент от Atlassian	Продвинутая аналитика, история покрытия

Интеграция

JaCoCo в CI/CD:

- Простая интеграция в облако
- Отчеты – XML
- Порог покрытия для прохождения билда
- GitHub Actions, GitLab
- *SonarQube*
- *JenKins*



JaCoCo gradle plugin for Android

Линейский Аким

